

TD/TP de synthèse — Un premier maillage 2D

Types, classes, RAII et règle de zéro avec Gmsh

MAM4 — Programmation C++ Durée : 2 heures

Objectif. Réutiliser les notions des quatre premières séances dans un petit programme scientifique qui produit un résultat visible. Le programme construira deux maillages du même rectangle à partir de deux configurations indépendantes.

Gmsh sert uniquement à fabriquer et à afficher le maillage. La fonction `generate_rectangle` qui dialogue avec son API est fournie : le travail porte sur le C++, pas sur les commandes propres à Gmsh.

Les sources sont fournies dans `code-td-synthese-gmsh.zip`. Après extraction, la commande `make` construit le programme `mesh_demo`. Ne pas employer `new` ou `delete` dans ce TD.

Exercice 0 — Obtenir immédiatement un maillage (10 min)

Objectif concret. Vérifier l'installation avant de modifier le programme.

Exécuter :

```
make
./mesh_demo
gmsh mesh.msh
```

Une fenêtre doit montrer un carré triangulé. En cas d'échec, relever la commande et le message exacts avant de demander de l'aide.

Lire rapidement les fichiers et répondre :

1. Quel fichier contient le `main` ?
2. Quelle fonction reçoit la configuration et produit le maillage ?
3. Quels fichiers contiennent les appels propres à Gmsh ?

À retenir. L'adaptateur Gmsh est fourni. Nous travaillerons sur les objets qui décrivent la configuration et la durée de vie de la session.

Exercice 1 — Lire ou modifier une configuration ? (15 min)

Objectif concret. Choisir entre `T`, `T&` et `const T&`.

Associer chaque intention à la signature correspondante :

```
void generate_rectangle(const MeshConfig& config);
void refine(MeshConfig& config, double mesh_size);
void print_config(const MeshConfig& config);
```

1. `generate_rectangle` doit-elle modifier la configuration ?
2. La modification effectuée par `refine` doit-elle rester visible chez l'appelant ?
3. Pourquoi `print_config` ne reçoit-elle pas une copie ?

Faire ensuite cette expérience volontaire :

```
void refine(const MeshConfig& config, double mesh_size)
{
    config.mesh_size = mesh_size;
}
```

Prédire le résultat, compiler, puis expliquer le diagnostic avec les verbes *lire* et *modifier*. Restaurer enfin la bonne signature.

À retenir. `T&` permet de modifier l'objet fourni. `const T&` permet de l'observer sans le copier ni le modifier par cette référence.

Exercice 2 — Empêcher une configuration absurde (25 min)

Objectif concret. Transformer une structure publique en classe qui maintient son invariant.

Avec la structure initiale, ces instructions compilent :

```
config.width = -2.0;
config.mesh_size = 0.0;
```

Elles ne décrivent pourtant aucun maillage valable. Écrire les quatre règles qui doivent rester vraies pour `width`, `height`, `mesh_size` et `output`. Ces règles constituent l'invariant de `MeshConfig`.

Transformer ensuite la structure en classe :

```
class MeshConfig {
public:
    MeshConfig(double width, double height,
               double mesh_size, std::string output);

    double width() const noexcept;
    double height() const noexcept;
    double mesh_size() const noexcept;
    const std::string& output() const noexcept;

    void set_mesh_size(double mesh_size);
    void set_output(std::string output);

private:
    double width_;
    double height_;
    double mesh_size_;
    std::string output_;
};
```

Le constructeur refuse une dimension ou une taille de maille inférieure ou égale à zéro, ainsi qu'un nom de fichier vide. Les deux setters doivent eux aussi maintenir l'invariant. Utiliser `std::invalid_argument` pour signaler une valeur refusée.

Adapter enfin `refine`, `print_config` et l'adaptateur fourni : ils doivent maintenant passer par l'interface publique de la classe.

Tester une configuration valide, puis essayer séparément une largeur négative, une taille de maille nulle et un nom vide. Retirer les essais invalides une fois le diagnostic vérifié.

Question. `private` garantit-il à lui seul que l'invariant sera respecté ? Qui doit vérifier les valeurs dans le constructeur et les setters ?

À retenir. `private` empêche le code extérieur de contourner l'interface. Le développeur de la classe reste responsable du maintien de l'invariant dans ses fonctions membres.

Exercice 3 — Générer un rectangle (10 min)

Objectif concret. Utiliser une classe correcte pour produire un premier résultat.

Remplacer la configuration initiale de `main` par :

```
MeshConfig config{2.0, 1.0, 0.25, "coarse.msh"};

print_config(config);
generate_rectangle(config);
```

Compiler et exécuter, puis ouvrir `coarse.msh`. Repérer le rectangle, les triangles et l'effet du paramètre `mesh_size`.

Question. Pourquoi `generate_rectangle` reçoit-elle `const MeshConfig&` plutôt que `MeshConfig` ou `MeshConfig&` ?

À retenir. L'adaptateur observe une configuration existante sans la copier et sans la modifier.

Exercice 4 — Lier la session Gmsh à la durée de vie d'un objet (20 min)

Objectif concret. Remplacer les appels manuels à `initialize` et `finalize` par un objet RAII.

Le programme de départ contient :

```
gmsh::initialize();
// generation du maillage
gmsh::finalize();
```

Que se passerait-il si un `return` était ajouté avant `gmsh::finalize()` ? Compléter la classe suivante :

```
class GmshSession {
public:
    GmshSession();
    ~GmshSession();

    GmshSession(const GmshSession&) = delete;
    GmshSession& operator=(const GmshSession&) = delete;
};
```

Le constructeur appelle `gmsh::initialize()` et le destructeur appelle `gmsh::finalize()`. Dans le fichier d'implémentation, ajouter :

```
#include <gmsh.h>
```

Le début du programme devient alors :

```
#include "gmsh_session.hpp"

int main()
{
    GmshSession session;
    // configurations et generations
}
```

Supprimer du `main` les deux appels manuels. Ajouter temporairement une trace dans le constructeur et le destructeur, prédire leur ordre, puis exécuter. Retirer ensuite les traces.

1. Qui appelle explicitement le destructeur de `session` ?
2. Pourquoi la copie de `GmshSession` est-elle interdite ?
3. Quelle différence avec la copie d'une simple configuration ?

À retenir. La session Gmsh suit la durée de vie de l'objet : c'est le RAII. Une session ne devant pas être dupliquée, sa copie est explicitement interdite.

Exercice 5 — Une copie, deux maillages indépendants (20 min)

Objectif concret. Vérifier la sémantique de valeur et la règle de zéro de `MeshConfig`.

Avant de compiler, répondre : devons-nous écrire nous-mêmes un constructeur de copie pour `MeshConfig` ? Combien d'objets existeront après `MeshConfig fine{coarse}` ?

Écrire :

```
MeshConfig coarse{2.0, 1.0, 0.25, "coarse.msh"};
MeshConfig fine{coarse};

refine(fine, 0.08);
fine.set_output("fine.msh");

generate_rectangle(coarse);
generate_rectangle(fine);
```

Ajouter des assertions ou des affichages qui vérifient :

- `coarse` conserve la taille de maille 0.25 et le nom `coarse.msh` ;
- `fine` utilise 0.08 et `fine.msh` ;
- les deux objets décrivent toujours le même rectangle.

Ouvrir successivement les deux fichiers. Les domaines doivent être identiques, mais le maillage `fine.msh` doit être plus dense.

Question. Pourquoi aucune opération spéciale de copie, de déplacement ou de destruction n'est-elle nécessaire dans `MeshConfig` ?

À retenir. Les `double` et `std::string` savent déjà se copier, se déplacer et se détruire. Leur composition donne naturellement à `MeshConfig` une copie indépendante : c'est la règle de zéro.

Exercice 6 — Bilan (10 min)

Compléter le tableau avec une ligne de code ou un nom de classe tiré du projet.

Notion	Où apparaît-elle dans le programme ?
<code>const T&</code>	
<code>T&</code>	
<code>private</code> et <code>invariant</code>	
constructeur	
destructeur et <code>RAII</code>	
copie indépendante	
copie interdite	
règle de zéro	

Répondre en une ou deux phrases :

1. Quel invariant protège `MeshConfig` ?
2. Pourquoi `GmshSession` possède-t-elle un destructeur ?
3. Pourquoi modifier `fine` ne modifie-t-il pas `coarse` ?
4. Pourquoi peut-on copier `MeshConfig`, mais pas `GmshSession` ?
5. Où reconnaît-on la règle de zéro ?

Signatures expressives $\longrightarrow$ **invariant** $\longrightarrow$ **RAII** $\longrightarrow$ **copie indépendante** $\longrightarrow$ **règle de zéro**.

Pour aller plus loin — facultatif

Placer plusieurs configurations dans :

```
std::vector<MeshConfig>
```

Les parcourir pour produire plusieurs maillages. Chaque configuration doit conserver son propre nom et sa propre taille de maille.